

```

// main.cpp

#include ".common/comm.h"
#include "CommandInterpreter.h"

#include <Arduino.h>
#include <RF24.h>
#include <SPI.h>

RF24 radio(9, 10); // CE, CSN

void setup()
{
    d_SerialBegin(9600);

    pinMode(Nano_Pins::ALC_SENSOR_PIN, INPUT);
    pinMode(Nano_Pins::CH3_SENSOR_PIN, INPUT);

    pinMode(Nano_Pins::FRESH_PIN, OUTPUT);
    pinMode(Nano_Pins::EXPIRED_PIN, OUTPUT);
    pinMode(Nano_Pins::GOING_BAD_PIN, OUTPUT);

    delay(1000);
    radio.begin();
    radio.setPALevel(RF24_PA_LOW);
    radio.setDataRate(RF24_250KBPS);
    radio.setChannel(76);
    radio.setRetries(5, 15);
    radio.openReadingPipe(1, MY_ADDRESS);
    radio.startListening();

    d_SerialPrintGroupLn(radio.isChipConnected() ? "Radio initialized." : "Radio
Uninitailized. Restart!");
    d_SerialPrintLn("Nano ready!");

    d_SerialPrint("My Address:");
    PrintAddress(MY_ADDRESS);
    d_SerialPrint("Main Address:");
    PrintAddress(MAIN_ADDRESS);
    d_SerialPrint("Packet size: ");
    d_SerialPrintLnV(sizeof(Packet));
    delay(1000);

    NanoInit(5); // Time Only, until we get an update from the UNO
}

void loop()
{
    NanoLoop();

    CommandInterpreter(&radio);
}

```

```

// CommandInterpreter.h

#include ".common/comm.h"

#include <stdint.h>

#define BOX_ID "A1"

constexpr uint64_t MAIN_ADDRESS = 0xF0F0F0AA11LL;
constexpr uint64_t MY_ADDRESS = CalcAddress(BOX_ID);

#include "RF24.h"
#include <Arduino.h>

enum Nano_Pins
{
    FRESH_PIN = 8,
    GOING_BAD_PIN = 7,
    EXPIRED_PIN = 6,
    ALC_SENSOR_PIN = A0,
    CH3_SENSOR_PIN = A1
};

// Prints given address in proper base64 format.
void PrintAddress(uint64_t addr);

// Similar to the UN0, just sets global values on the Nano according to the thresholds.
void EvaluateCondition();

// Simple caller function to set thresholds.
void NanoInit(uint8_t unit_profile);

// Code meant to be called every loop.
void NanoLoop();

// Function to write back to the UN0.
void NanoNotify(RF24 *radio);

// Function meant to handle all radio input.
void CommandInterpreter(RF24 *radio);

// CommandInterpreter.cpp
#include <.common/comm.h>
#include <stdint.h>

#include "Arduino.h"
#include "CommandInterpreter.h"
#include "RF24.h"

ThresholdValues thresholds;
FoodCondition CurrentCondition;
Packet response_packet;

```

```

void PrintAddress(uint64_t addr)
{
    d_SerialPrint(" 0x");

    for (int i = 4; i >= 0; i--) // nRF24 uses 5-byte addresses
    {
        uint8_t byte = (addr >> (8 * i)) & 0xFF;

        if (byte < 0x10)
            d_SerialPrintV('0');

        d_SerialPrintGroup(byte, HEX);
    }
    d_SerialPrintln("");
}

void CommandInterpreter(RF24 *radio)
{
    radio->stopListening();
    radio->openReadingPipe(1, MY_ADDRESS);
    radio->startListening();

    if (!radio->available())
        return;

    Packet request;
    radio->read(&request, sizeof(request));

    uint8_t type, request_sequence;
    FoodCondition condition;
    uint8_t additional_info;

    if (!UnpackPacket(request, type, request_sequence, condition, additional_info))
    {
        Serial.println("Nano: bad request packet");
        return;
    }

    switch (type)
    {
    case PacketType::PKT_CONFIG:
        d_SerialPrint("Given Profile ID: ");
        d_SerialPrintlnV(additional_info);
        NanoInit(additional_info);
        break;
    case PacketType::PKT_REQUEST:
    {
        d_SerialPrint("Given Profile ID: ");
        d_SerialPrintlnV(additional_info);
        NanoInit(additional_info);

        radio->stopListening();
    }
    }
}

```

```

        PackPacket(response_packet, PacketType::PKT_RESPONSE, request_sequence,
CurrentCondition);
        d_SerialPrint("Writing to main box, address:");
        PrintAddress(MAIN_ADDRESS);
        delayMicroseconds(150);
        radio->openWritingPipe(MAIN_ADDRESS);

        bool ok = radio->write(&response_packet, sizeof(response_packet));
        d_SerialPrintlnV(ok ? "Request sent!" : "Request failed!");
    }
    break;
case PacketType::PKT_RESPONSE:
    d_SerialPrintln("Silly Arduino Uno! You have sent the wrong message");
    break;
}
radio->flush_rx();
}

void EvaluateCondition()
{
    int alc_reading = analogRead(Nano_Pins::ALC_SENSOR_PIN);
    int ch3_reading = analogRead(Nano_Pins::CH3_SENSOR_PIN);

    d_SerialPrint("Alcohol Values: ");
    d_SerialPrintlnV(alc_reading);
    d_SerialPrint("CH3 Values : ");
    d_SerialPrintlnV(ch3_reading);

    if (alc_reading >= thresholds.alc_exp || ch3_reading >= thresholds.ch3_exp)
    {
        d_SerialPrintln("\nExpired!\n\n");
        digitalWrite(Nano_Pins::EXPIRED_PIN, HIGH);
        digitalWrite(Nano_Pins::FRESH_PIN, LOW);
        digitalWrite(Nano_Pins::GOING_BAD_PIN, LOW);
        d_SerialPrint("Current Threshold Value is:\nAlc:");
        d_SerialPrintlnV(thresholds.alc_exp);
        d_SerialPrint("ch3:");
        d_SerialPrintlnV(thresholds.ch3_exp);
        CurrentCondition = FoodCondition::EXPIRED;
    }
    else if (alc_reading >= thresholds.alc_bad || ch3_reading >= thresholds.ch3_bad)
    {
        d_SerialPrintln("\nGoing Bad!\n\n");
        digitalWrite(Nano_Pins::GOING_BAD_PIN, HIGH);
        digitalWrite(Nano_Pins::FRESH_PIN, LOW);
        digitalWrite(Nano_Pins::EXPIRED_PIN, LOW);
        d_SerialPrint("Current Threshold Value is:\nAlc:");
        d_SerialPrintlnV(thresholds.alc_bad);
        d_SerialPrint("ch3:");
        d_SerialPrintlnV(thresholds.ch3_bad);
        CurrentCondition = FoodCondition::GOING_BAD;
    }
    else

```

```
{
    d_SerialPrintln("\nFresh!\n\n");
    digitalWrite(Nano_Pins::FRESH_PIN, HIGH);
    digitalWrite(Nano_Pins::GOING_BAD_PIN, LOW);
    digitalWrite(Nano_Pins::EXPIRED_PIN, LOW);
    CurrentCondition = FoodCondition::FRESH;
}

void NanoInit(uint8_t unit_profile)
{
    thresholds = profile_table[unit_profile];
}

void NanoLoop()
{
    EvaluateCondition();
}

void NanoNotify(RF24 *radio)
{
    if (!(radio->available()))
        d_SerialPrintln("Radio is not available!");
}
```